Embedding Equations of the Small Language Model

This section gives the equations of every embedding used by the system and of the layers that read them. Both trained networks share the same 95-entry token vocabulary: the 91 words of the lookup table and four special tokens. The word picker (Case 2, hearing to deaf-mute) embeds each spoken word together with its position in the sentence, whereas TinyLlama (Case 1, deaf-mute to hearing) embeds the recognized signs without any position and adds position only to the English words it writes. Table 1 lists the symbols.

Symbol	Meaning	Value
V	token vocabulary (91 words + <pad>, <bos>, <eos>, <unk>)	$\|V\| = 95$
d	embedding size (numbers per token)	64
L	largest number of words or signs in one turn	16
h, d_h	attention heads, size of one head (d/h)	4, 16
E_w, P	word picker: word table, position table	95 × 64, 16 × 64
E_s, E_t	TinyLlama: sign table, table of the words it writes	95 × 64 each
t_i	token ID of the i-th word or sign	0 … 94
n, k	number of spoken words, number of signs	≤ 16
M	sign memory read by TinyLlama	k × 64

Table 1. Symbols used in the equations.

A. From words to vectors

Every word or sign is first replaced by its token ID (Equation 1). An embedding lookup then reads one row of a learned table; this is the same as multiplying a one-hot vector by the table (Equation 2), so the embedding is a learned linear map from the vocabulary to a 64-dimensional space. The row of the padding token is fixed at zero and is never trained (Equation 3).

$$t_i = \mathrm{id}(w_i) \in \{0,1,\ldots,94\} \tag{1}$$

$$e(w) = E^{\top} o(w) = E[\mathrm{id}(w)], \qquad o(w) \in \{0,1\}^{95} \tag{2}$$

$$E[0] = 0 \ \ (\text{ID 0 is the padding token}) \tag{3}$$

B. Case 2: the word picker (word and position)

For the hearing person's words, the embedding is the sum of a word vector and a learned position vector (Equation 4), normalized by layer normalization (Equations 5 and 6). Two pre-normalized transformer blocks then let every word attend to every other word (Equations 7 to 10); padding positions receive a score of minus infinity so they are never attended to (Equation 8). The tag head scores, for every word, a drop choice and sixteen keep choices (Equation 11). Since live testing, only the keep-or-drop decision of Equation 12 is used, and only for the signs my and you; the question head decides from the average word vector whether the marker how is added (Equation 13).

$$x_i = E_w[t_i] + P[i], \qquad i = 1, \dots, n \le L \tag{4}$$

$$z_i = \mathrm{LN}(x_i) = \gamma \odot \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} + \beta, \qquad \varepsilon = 10^{-12} \tag{5}$$

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij}\,, \qquad \sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}\left(x_{ij} - \mu_i\right)^2 \tag{6}$$

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_h}} + B\right)V \tag{7}$$

$$B_{ij} = \begin{cases} 0 & \text{if word } j \text{ is real} \\ -\infty & \text{if word } j \text{ is padding} \end{cases} \tag{8}$$

$$Z' = Z + \mathrm{MHA}\big(\mathrm{LN}(Z)\big), \qquad Z'' = Z' + \mathrm{FFN}\big(\mathrm{LN}(Z')\big) \tag{9}$$

$$\mathrm{FFN}(u) = W_2\,\mathrm{GELU}(W_1 u + b_1) + b_2 \tag{10}$$

$$s_i = W_{\mathrm{tag}} z_i + b_{\mathrm{tag}} \in \mathbb{R}^{17}, \qquad s_i = (s_{i0}, s_{i1}, \dots, s_{i16}) \tag{11}$$

$$\mathrm{keep}_i \Leftrightarrow \max_{r \ge 1} s_{ir} > s_{i0} \text{ (used only for my, you)} \tag{12}$$

$$q = W_q\left(\frac{1}{n}\sum_{i=1}^{n} z_i\right) + b_q \in \mathbb{R}^2, \text{ add how if } q_1 > q_0 \tag{13}$$

C. Case 1: TinyLlama (signs without position, words with rotary position)

Each recognized sign is embedded by the sign table alone, with no position vector (Equation 14). When Active Context supplies a remembered person, that person's vector is placed in front of the signs (Equation 15). The decoder reads the signs only through cross-attention (Equation 16), a weighted sum over the signs. Because the sum and the softmax do not depend on the order of their terms, the result is the same for every ordering of the signs (Equation 17). The English words TinyLlama writes are embedded by a second table (Equation 18) and receive their position through rotary position embedding (Equations 19 to 21), applied to the queries and keys of the causal self-attention, which can only look at earlier words (Equation 22). Each decoder block uses root mean square normalization (Equation 23), causal self-attention, cross-attention to the signs and a SwiGLU feed-forward layer (Equations 24 to 27). The output head scores the 95 tokens and the highest score is written (Equations 28 and 29).

$$m_j = E_s[t_j], \qquad M = [m_1; m_2; \ldots; m_k] \in \mathbb{R}^{k\times 64} \tag{14}$$

$$M' = [E_s[t_{\text{person}}]; M] \tag{15}$$

$$c = \sum_{j=1}^{k} \alpha_j v_j\,, \qquad \alpha_j = \frac{e^{a_j}}{\sum_{l=1}^{k} e^{a_l}}, \qquad a_j = \frac{q \cdot W_K m_j}{\sqrt{d_h}}, \qquad v_j = W_V m_j \tag{16}$$

$$\text{CrossAttn}(H, \Pi M) = \text{CrossAttn}(H, M) \text{ for every reordering } \Pi \text{ of the signs} \tag{17}$$

$$h_t^{(0)} = E_t[y_t] \tag{18}$$

$$\begin{pmatrix} q'_{2r} \\ q'_{2r+1} \end{pmatrix} = \begin{pmatrix} \cos m\theta_r & -\sin m\theta_r \\ \sin m\theta_r & \cos m\theta_r \end{pmatrix} \begin{pmatrix} q_{2r} \\ q_{2r+1} \end{pmatrix} \tag{19}$$

$$\theta_r = 10000^{-\frac{2r}{d_h}}, \qquad r = 0,1,\ldots,\frac{d_h}{2} - 1 \tag{20}$$

$$(R_m q)^{\top}(R_n k) = q^{\top} R_{n-m} k \tag{21}$$

$$C_{mn} = \begin{cases} 0 & \text{if } n \le m \\ -\infty & \text{if } n > m \end{cases} \tag{22}$$

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \varepsilon}} \odot g, \qquad \varepsilon = 10^{-6} \tag{23}$$

$$H' = H + \mathrm{SelfAttn}\big(\mathrm{RMSNorm}(H)\big) \tag{24}$$

$$H'' = H' + \mathrm{CrossAttn}(\mathrm{RMSNorm}(H'), M) \tag{25}$$

$$H''' = H'' + \mathrm{SwiGLU}\big(\mathrm{RMSNorm}(H'')\big) \tag{26}$$

$$\mathrm{SwiGLU}(x) = W_{\mathrm{down}}\big(\mathrm{SiLU}\big(W_{\mathrm{gate}}x\big) \odot W_{\mathrm{up}}x\big),\ \mathrm{SiLU}(a) = a \cdot \sigma(a) \tag{27}$$

$$P(\, y_{t+1} \mid y_{\le t}, M\,) = \mathrm{softmax}\big(W_o \mathrm{RMSNorm}(h_t)\big) \in \mathbb{R}^{95} \tag{28}$$

$$\hat{y}_{t+1} = \arg\max_{v \in V} P(\, v \mid y_{\le t}, M\,) \tag{29}$$

D. How the embedding values are learned

The numbers in the four tables are not set by hand. Every row is a parameter that is trained together with the rest of each network by minimizing cross-entropy: the word picker on its per-word labels and its question flag (Equation 30), and TinyLlama on predicting each next word of the reference sentence (Equation 31). Both use the AdamW update (Equation 32) with learning rate 0.001 and weight decay 0.01, for 100 epochs (word picker) and 150 epochs (TinyLlama), batch size 8 and a fixed random seed of 42.

$$\mathcal{L}_{\mathrm{picker}} = -\sum_{i=1}^{n} \log P(\, \ell_i \mid x\,) - \log P(\, q \mid x\,) \tag{30}$$

$$\mathcal{L}_{\mathrm{decoder}} = -\sum_{t} \log P(\, y_{t+1} \mid y_{\le t}, M\,) \tag{31}$$

$$\theta \leftarrow \theta - \eta\left(\frac{\widehat{m}}{\sqrt{\widehat{v}} + \epsilon} + \lambda\theta\right), \qquad \eta = 10^{-3}, \qquad \lambda = 0.01 \tag{32}$$

E. Reading the embedding space

The similarity of two learned vectors is measured with the cosine (Equation 33). Over the 34 signs, signs of the same category are only slightly more similar than signs of different categories: 0.05 against -0.01 in TinyLlama's sign table and 0.08 against 0.03 in the word picker's table, with no pair above 0.36. The tables are shaped by the two tasks rather than by topic, which is why the seven conversation contexts are tracked by the rule-based weights of Section F instead of by the embedding space.

$$\cos(u, v) = \frac{u \cdot v}{\|u\|\|v\|} \tag{33}$$

F. Context weights (the seven contexts)

Active Context keeps a weight for each of the seven contexts. In each turn, the share of a context is the confidence-weighted fraction of the turn's context words that belong to it, a word that fits the active context being counted only for that context (Equation 34). Every weight moves halfway toward its share, so a context leaks in when it is mentioned and fades when it is not (Equation 35). The active context changes only when another context covers at least half of the context words and more than the active one (Equation 36).

$$s_c = \frac{\sum_{w \in C_c} \mathrm{conf}(w)}{\sum_{w \in C} \mathrm{conf}(w)} \tag{34}$$

$$w_c \leftarrow w_c + \alpha(s_c - w_c), \qquad \alpha = 0.5 \tag{35}$$

$$T \leftarrow c^* \text{ if } s_{c^*} \geq 0.5 \text{ and } s_{c^*} > s_T, \qquad c^* = \arg\max_{c \neq T}(s_c, w_c) \tag{36}$$

Why These Solutions: Literature and Our Evidence

Table 2 gives, for every design choice in the equations, the published basis and the measurement taken on our own system. The accuracy figures compare the system output

with the dataset sentences, from an empty conversation memory, and the held-out test split is reported separately.

Choice (equation)	Basis in the literature	Our evidence
Learned lookup-table embeddings (2–3)	Words as learned dense vectors trained with the task (Bengio et al., 2003; Mikolov et al., 2013).	Each table holds only 6,080 numbers (95 × 64); TinyLlama reproduces 663/663 (100.0%) dataset sentences, test split 24/24 (100.0%).
Embedding size d = 64, 4 heads of 16 (all)	BERT-Tiny uses 2 layers and width 128 for a 30,522-token vocabulary (Turc et al., 2019); our vocabulary is 95 tokens, so half the width with the same depth suffices.	Whole models: 75,411 (word picker) and 133,376 (TinyLlama) numbers; hearing → deaf output 596/608 (98.0%), test split 172/178 (96.6%).
Word + position by addition (4)	The Transformer, BERT and DistilBERT add a position vector to each word vector (Vaswani et al., 2017; Devlin et al., 2019; Sanh et al., 2019).	Keep/drop decisions on all 2,497 labelled words: 2,491/2,497 (99.8%); with the position vectors shuffled 2,304/2,497 (92.3%), so the picker uses word position.
Learned, not sinusoidal, positions; L = 16 (4)	Learned and sinusoidal positions gave nearly identical results (Vaswani et al., 2017, Sec. 3.5); BERT and DistilBERT learn them. Sinusoids only help beyond the trained length, which never happens here.	Every turn is cut to 16 words before embedding (Algorithm 3), so no unseen position can occur.
Layer normalization after the sum, $\varepsilon = 10^{-12}$ (5–6)	Layer normalization (Ba et al., 2016); BERT normalizes its embeddings with the same ε.	Adds only 128 numbers (γ, β).
Pre-normalized blocks, GELU feed-forward (9–10)	Normalizing before each sub-layer trains stably without warm-up (Xiong et al., 2020); GELU is BERT's activation (Hendrycks & Gimpel, 2016).	Trained in 100 epochs with a constant learning rate, no warm-up schedule.
Scaled dot-product attention, $-\infty$ mask (7–8, 22)	Dividing by $\sqrt{d_h}$ keeps the softmax out of its flat region; masked scores are set to $-\infty$ (Vaswani et al., 2017).	Padding never changes a result; in the live demo there is one sentence and no padding (mask all false).
Only my / you left to the picker (12)	Design rule taken from the training data itself.	The dataset leaves out only my (100 times) and you (95 times); the picker had dropped "today" with a 97% drop score.
No position on the signs (14, 16–17)	Attention without position encoding is permutation-invariant (Zaheer et al., 2017; Lee et al., 2019).	All 629 multi-sign inputs give the same sentence in every order; largest score difference 3.8e-06 (rounding).
Person vector placed in front of the signs (15)	Cross-attention reads a set of vectors (Vaswani et al., 2017); adding one more vector adds the person.	"home" after "Where is your mom?" → "mother is home."

Choice (equation)	Basis in the literature	Our evidence
Rotary positions for the written words (19–21)	Rotary position embedding makes attention depend on relative distance (Su et al., 2024); used by LLaMA and TinyLlama (Touvron et al., 2023; Zhang et al., 2024).	Training sentences reproduced: 639/639 (100.0%).
RMSNorm, $\varepsilon = 10^{-6}$ (23)	Comparable accuracy to layer normalization with 7–64% less running time (Zhang & Sennrich, 2019).	One scale vector of 64 numbers per normalization.
SwiGLU feed-forward (27)	Gated linear units improve on ReLU and GELU feed-forward layers (Shazeer, 2020); SiLU (Elfwing et al., 2018).	Used as in TinyLlama.
Greedy decoding (29)	The highest-scoring token is chosen at each step; same signs → same sentence.	In the traced example every chosen word had about 100% of the probability, so a wider search would not change it.
Cross-entropy + AdamW, $\eta = 0.001$ (30–32)	AdamW decouples weight decay from the gradient step (Loshchilov & Hutter, 2019).	Fixed seed 42: training is reproducible.
Cosine similarity (33)	Standard measure of closeness of word vectors (Mikolov et al., 2013).	Shows the tables do not group signs by topic, so contexts are rule-based (Section F).
Context weights, $\alpha = 0.5$, switch at ½ (34–36)	Adviser's consultation: other contexts leak instead of dropping to 0; a whole-context change needs a majority of the turn (9 of 16).	Conversation test: topic correct 34/34; mixed contexts 7/7. $\alpha = 0.5$ is a design value still to confirm.

Table 2. Design choices of the embeddings and their support.

Where Each Equation Is in the Code

Equations	File and function
1	tokenizer.py: SimpleTokenizer.token_to_id; slm_pipeline.py: tokenize()
2–6	encoder_distilbert.py: DistilBertEmbeddings (word_embeddings, position_embeddings, layer_norm)
7–10	encoder_distilbert.py: MultiHeadSelfAttention, FFN, TransformerBlock
11–13	word_picker.py: WordPicker.heads(); restrict_to_signs()
14	decoder_tinyllama.py: TinyLlamaForGeneration.src_embedding, embed_source()
15	slm_pipeline.py: deaf_to_non_deaf() (torch.cat after fill_subject())
16–17	decoder_tinyllama.py: CrossAttention
18–22	decoder_tinyllama.py: token_embedding, precompute_rope_freqs(), apply_rope(), causal_mask
23–27	decoder_tinyllama.py: RMSNorm, TinyLlamaCausalAttention, TinyLlamaBlock, SwiGLU
28–29	decoder_tinyllama.py: output_head, decode(), generate_from_memory()

Equations	File and function
30–32	train_picker.py, train_tinyllama.py: CrossEntropyLoss, torch.optim.AdamW
34–36	slm_pipeline.py: ActiveContext.update_topic(), TOPIC_SWITCH_SHARE, TOPIC_FADE

References

Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer normalization*. arXiv:1607.06450.

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research, 3*, 1137–1155.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186).

Elfwing, S., Uchibe, E., & Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks, 107*, 3–11.

Hendrycks, D., & Gimpel, K. (2016). *Gaussian error linear units (GELUs)*. arXiv:1606.08415.

Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., & Teh, Y. W. (2019). Set Transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of ICML 2019* (pp. 3744–3753).

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of ICLR 2019*.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv:1301.3781.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*. arXiv:1910.01108.

Shazeer, N. (2020). *GLU variants improve Transformer*. arXiv:2002.05202.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing, 568*, 127063.

Touvron, H., et al. (2023). *LLaMA: Open and efficient foundation language models*. arXiv:2302.13971.

Turc, I., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *Well-read students learn better: On the importance of pre-training compact models*. arXiv:1908.08962.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30*.

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T.-Y. (2020). On layer normalization in the Transformer architecture. In *Proceedings of ICML 2020*.

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., & Smola, A. (2017). Deep sets. In *Advances in Neural Information Processing Systems 30*.

Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. In *Advances in Neural Information Processing Systems 32*.

Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). *TinyLlama: An open-source small language model*. arXiv:2401.02385.